ASSIGNMENT 9

Joins and Subqueries

Event Ticketing and Venue Seating System

Name	Aishani Dutta
Registration Number	2511021240020
Course / Section	BCA SEM 3 2025
Date	23 October 2026

These reports combine customer, event, booking and payment details. They use the same records from Assignment 7, so the totals can be checked against the earlier queries.

```
USE EventTicketingDB;
```

Booking list with customer and event

```
SELECT b.booking_id, u.name, e.title, b.booking_status
FROM Bookings b
JOIN Users u ON u.user_id = b.user_id
JOIN Events e ON e.event_id = b.event_id
ORDER BY b.booking_id;
```

Output

booking_id	name	title	booking_status
201	Ananya	Acoustic Evening	Confirmed
202	Riya	Acoustic Evening	Confirmed
203	Soham	The Last Letter	Confirmed
204	Ishaan	The Last Letter	Pending
205	Ananya	Design Meetup	Confirmed
206	Mitali	Jazz Night	Confirmed
207	Tanya	Jazz Night	Pending
208	Riya	Design Meetup	Confirmed
209	Soham	Jazz Night	Cancelled
210	Ishaan	Acoustic Evening	Confirmed

The joins replace user and event identifiers with readable names.

Bookings for every event

```
SELECT e.event_id, e.title, COUNT(b.booking_id) AS bookings
FROM Events e
LEFT JOIN Bookings b ON b.event_id = e.event_id
GROUP BY e.event_id, e.title
ORDER BY e.event_id;
```

Output

event_id	title	bookings
101	Acoustic Evening	3
102	The Last Letter	2
103	Design Meetup	2
104	Jazz Night	3
105	Comedy Hour	0

LEFT JOIN keeps Comedy Hour in the report even though it has no bookings. COUNT(b.booking_id) gives zero for that event.

Available seats for Acoustic Evening

```
SELECT s.seat_id, s.row_label, s.seat_number, s.seat_type
FROM Seats s
JOIN Events e ON e.venue_id = s.venue_id
WHERE e.event_id = 101
  AND NOT EXISTS (
      SELECT 1 FROM Booking_Seats bs
      WHERE bs.event_id = e.event_id
        AND bs.seat_id = s.seat_id
  )
ORDER BY s.seat_id;
```

Output

seat_id	row_label	seat_number	seat_type
3	A	3	Premium
6	B	3	Regular

Seats 3 and 6 are available. The subquery checks the selected event only, since a physical seat can be used again for a different event.

Customers without a booking

```
SELECT u.user_id, u.name
FROM Users u
WHERE NOT EXISTS (
    SELECT 1 FROM Bookings b WHERE b.user_id = u.user_id
)
ORDER BY u.user_id;
```

Output

user_id	name
7	Nikhil
8	Pooja

Nikhil and Pooja have registered but have not made any bookings.

Bookings above the confirmed-booking average

```
SELECT booking_id, total_amount
FROM Bookings
WHERE booking_status = 'Confirmed'
  AND total_amount > (
      SELECT AVG(total_amount) FROM Bookings
      WHERE booking_status = 'Confirmed'
  )
ORDER BY total_amount DESC, booking_id;
```

Output

booking_id	total_amount
206	1200.00
201	1000.00
203	800.00

The confirmed-booking average is approximately Rs. 628.57. Bookings 206, 201 and 203 are above it.

Successful collections by event

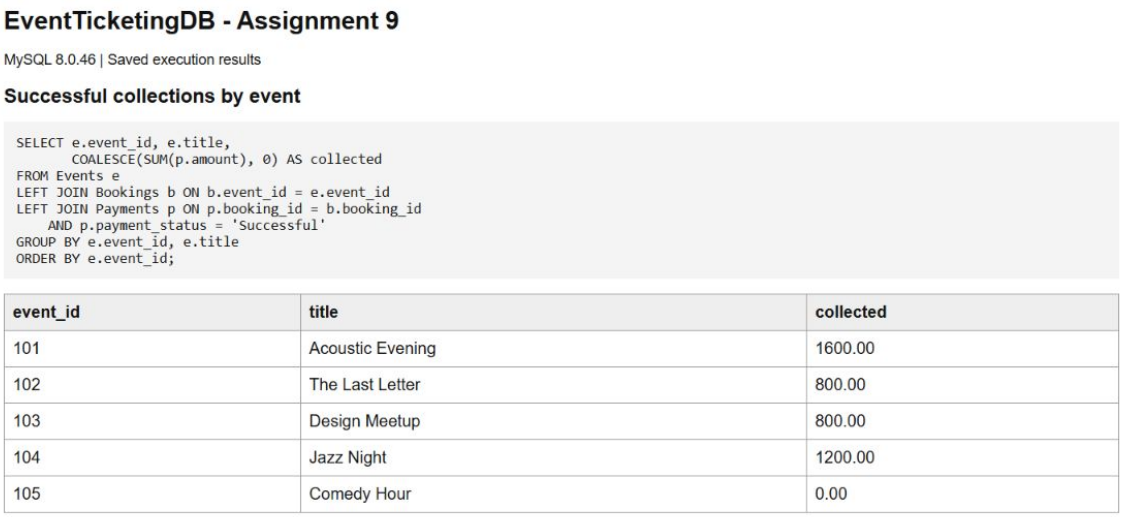
```
SELECT e.event_id, e.title,
       COALESCE(SUM(p.amount), 0) AS collected
FROM Events e
LEFT JOIN Bookings b ON b.event_id = e.event_id
LEFT JOIN Payments p ON p.booking_id = b.booking_id
    AND p.payment_status = 'Successful'
GROUP BY e.event_id, e.title
ORDER BY e.event_id;
```

Output

event_id	title	collected
101	Acoustic Evening	1600.00
102	The Last Letter	800.00
103	Design Meetup	800.00
104	Jazz Night	1200.00
105	Comedy Hour	0.00

The total collected is Rs. 4,400. The failed attempt for booking 202 is excluded. Booking_Seats is not joined here, because doing so would repeat a payment once for each seat.

Query result screenshot



EventTicketingDB - Assignment 9

MySQL 8.0.46 | Saved execution results

Successful collections by event

```
SELECT e.event_id, e.title,
       COALESCE(SUM(p.amount), 0) AS collected
FROM Events e
LEFT JOIN Bookings b ON b.event_id = e.event_id
LEFT JOIN Payments p ON p.booking_id = b.booking_id
    AND p.payment_status = 'Successful'
GROUP BY e.event_id, e.title
ORDER BY e.event_id;
```

event_id	title	collected
101	Acoustic Evening	1600.00
102	The Last Letter	800.00
103	Design Meetup	800.00
104	Jazz Night	1200.00
105	Comedy Hour	0.00

Saved MySQL results displayed in the browser.